Toward a Fair Evaluation Protocol for Automatic Prompt Optimization Methods

Raul José Silvério da Silva[1], **Aurora Trinidad Ramirez Pozo**[1]

[1]Programa de Pós-Graduação em Informática (PPGInf)
Universidade Federal do Paraná (UFPR) – Curitiba, PR – Brazil

raul.silverio@ufpr.br, aurora@inf.ufpr.br

Abstract. *Large Language Models are highly sensitive to prompt phrasing, motivating many Automatic Prompt Optimization (APO) methods that search the prompt space without gradients. Comparing them fairly is hard because papers report results under different budgets, datasets, seeds, and base models. We introduce* APOBENCH*, a five-pillar evaluation protocol standardizing budget allocation, seed control, benchmark diversity, multi-metric reporting, and model parity. Validating it on seven gradient-free methods across BBH, GSM8K, and HumanEval, we find that no method dominates: the lightweight ones use up to 11× fewer tokens than the most accurate while losing little accuracy, and the stop reason exposes convergence differences that accuracy alone hides.*

Keywords: Large Language Models, Prompt Optimization, Automated Machine Learning.

1. Introduction

Large Language Model (LLM) performance on a given task depends strongly on how that task is expressed as a prompt [Brown et al. 2020, Wei et al. 2022]. A prompt that achieves high accuracy on a reasoning task can be substantially outperformed by a differently phrased but semantically equivalent instruction, a phenomenon broadly referred to as prompt sensitivity [Sclar et al. 2024, Mizrahi et al. 2024, Zhuo et al. 2024].

Automatic Prompt Optimization (APO) addresses this challenge by searching the prompt space without gradient access. Since APE [Zhou et al. 2023], the first LLM-driven gradient-free optimizer, dozens of methods have appeared, with an evolutionary branch emerging in 2023 [Guo et al. 2024, Fernando et al. 2024] alongside search- and feedback-based methods [Yang et al. 2024, Pryzant et al. 2023]; surveys catalog them across paradigms [Ramnath et al. 2025, Li et al. 2025].

In practice, however, comparing APO methods fairly is impractical, because papers report results on different datasets, under different budgets, often from a single random seed and against different base models. Accuracies obtained under incomparable budgets or a single seed cannot be fairly ranked, yet few papers acknowledge the trade-off.

Established frameworks exist for evaluating language *models* (HELM [Liang et al. 2023], LM-Eval-Harness [Gao et al. 2024]), but no analogous standard exists for the APO *methods* that produce prompts. Emerging tools [Zhu et al. 2023, Zehle et al. 2026, Zheng et al. 2025] address parts of the problem, but none standardizes budgets, seeds, benchmarks, and metrics jointly.

This paper makes four contributions:

1. **APOBENCH protocol.** We define a five-pillar evaluation framework for fair and reproducible comparison of APO methods, formalizing decisions that existing work leaves implicit: budget allocation, seed control, benchmark diversity, multi-metric reporting, and model parity.
2. **Empirical study.** We compare seven gradient-free APO methods (SEE, GAAPO, PromptBreeder, CriSPO, DSGE-Evo, PLUM and HPSS) across 10 tasks spanning three benchmark categories (BBH, GSM8K, HumanEval), using a single fixed model and identical experimental conditions.
3. **Multi-dimensional analysis.** We report accuracy, token cost, wall-clock time, convergence behavior (generations completed and stop reason), and stability across seeds.
4. **Open implementation.** We release APOBENCH as an open-source package containing the optimizers, the fixed splits, the run configurations and the aggregation and statistical-comparison scripts, so that a new method can be scored against these entries under identical rules (Section 5.3).

Under our fixed budget and base model, no method dominates all benchmarks, and those ranking first on accuracy can rank last on efficiency.

2. Background

LLMs are neural language models pre-trained on large corpora [Brown et al. 2020] that exhibit emergent capabilities at sufficient scale [Wei et al. 2022]. Downstream performance depends strongly on how a task is communicated through the prompt: small lexical changes can produce accuracy swings of up to 76 percentage points, averaging roughly 10 points across a broad task suite [Sclar et al. 2024].

APO formalizes prompt search as a discrete, budget-constrained optimization problem [Ramnath et al. 2025]; gradient-free methods such as APE [Zhou et al. 2023] approximate it without access to gradient information. Let $\mathcal{P}$ denote the space of natural-language prompt strings, D_{val} a fixed validation set, and $\mathcal{F}(p, D_{\text{val}})$ a scalar performance metric (accuracy, F1, or pass@1) for prompt p. The optimization objective is:

$$p^* = \arg\max_{p \in \mathcal{P}} \mathcal{F}(p, D_{\text{val}}) \quad \text{subject to} \quad \text{cost}(p) \leq B, \tag{1}$$

where B is a resource budget (token count, API calls, or wall-clock time) and cost$(\cdot)$ aggregates resource consumption during the search. Access to the model is *black-box*: only text outputs are observed, with no gradient information available.

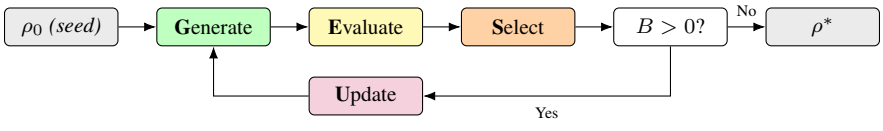

Figure 1. The GESU optimization cycle proposed in APOBENCH.

An APO method implements Equation 1 through the iterative Generate–Evaluate–Select–Update (GESU) cycle of Figure 1 [Ramnath et al. 2025]: generate candidates, evaluate them on D_{val}, select the top-k, and update the search state until the budget B is exhausted. Methods differ primarily in how they generate and update candidates.

3. Related Work

3.1. Frameworks for Evaluating Language Models

The community has produced widely adopted frameworks for evaluating LLMs: HELM [Liang et al. 2023] covers 42 scenarios across seven core metrics; the LM Evaluation Harness [Gao et al. 2024] provides a unified zero-shot/few-shot interface powering public leaderboards; and BIG-Bench Hard [Suzgun et al. 2023] curates 23 challenging reasoning tasks. All evaluate *models* under fixed prompts, not the optimization *process* or the APO methods that produce them.

3.2. APO Method Surveys and Comparison Tools

Surveys have cataloged the APO landscape without standardizing its evaluation. Ramnath et al. [Ramnath et al. 2025] decompose APO methods into five stages (seed initialization, evaluation and feedback, candidate generation, filtering and iteration control) and categorize the literature along them. Li et al. [Li et al. 2025] survey APO from an optimization-theoretic lens, identifying cross-paradigm fragmentation as the primary gap.

Comparison tools each address only part of the problem. `promptbench` [Zhu et al. 2023] stress-tests prompts against adversarial perturbations rather than studying optimizers; **GreaterPrompt** [Zheng et al. 2025] unifies several APO methods behind one API but treats evaluation as incidental; and **DSPy** [Khattab et al. 2024] compiles declarative pipelines with optimization operators, making its contribution architectural, not evaluative. The closest precedent is `promptolution` [Zehle et al. 2026], which runs CAPO [Zehle et al. 2025], EvoPromptDE/GA, and OPRO under a unified configuration with token tracking, and whose companion benchmark compares three frameworks on SST-5 and GSM8K under matched token budgets. **EvoPrompt** [Guo et al. 2024] is comparable to GAAPO and PromptBreeder but is excluded here pending model-parity alignment.

ProTeGi [Pryzant et al. 2023] (EMNLP 2023) frames instruction optimization as "gradient descent in text space" via beam search over LLM-generated critiques, making it the direct ancestor of CriSPO's critique-rewrite design. It requires per-example error labels, which are unavailable in APOBENCH's zero-shot setup, so we treat it as out of scope for this study.

Recent frameworks address parts of this challenge. Promptolution standardizes token budgets within one ecosystem, and GreaterPrompt unifies method APIs. None of them, however, jointly mandates all five dimensions of budget, seed control, benchmark diversity, multi-metric reporting and model parity. APOBENCH assembles these requirements into a single portable protocol, enabling head-to-head comparisons that existing work does not support.

4. The APOBENCH Evaluation Protocol

APOBENCH is organized around five pillars. Each pillar makes explicit a decision that previous work has left implicit or inconsistent, and each is necessary for a fair comparison.

Pillar 1 — Budget Standardization. APOBENCH defines three complementary limits applied simultaneously: a *wall-clock time cap* T (3,600 s per optimizer–task pair in this work), an *API-call cap* C (5,000 calls), and a *token cap* K (5,000,000 tokens).

All limits and the actual resource consumed must be reported, together with the *stop reason* recording whether the optimizer converged, reached early stopping or exhausted its budget. A budget-exhausted score is a lower bound rather than a final result.

Pillar 2 — Seed Control. Single-run results are unreliable because APO methods involve stochastic search. APOBENCH requires a minimum of three fixed random seeds per optimizer–task pair, with mean and standard deviation reported for every cell; single-run numbers are never reported. Results with standard deviation exceeding 0.05 on a given task should be explicitly flagged as high-variance. We recommend five seeds when computational budget allows.

Reporting variance is necessary but not sufficient, since a protocol must also fix when two methods count as different. All methods are measured on the same task set, so comparisons are naturally paired, and APOBENCH prescribes a two-sided paired t-test over the per-task differences, reported with its mean difference and win–loss record. We prefer it to a sign test, which discards effect sizes. Since m methods yield $m(m-1)/2$ hypotheses, a Holm–Bonferroni correction across that family is also required.

Pillar 3 — Benchmark Diversity. No single benchmark captures the full spectrum of task types. APOBENCH requires a suite covering multiple cognitive dimensions, here multi-step reasoning (BBH [Suzgun et al. 2023]), mathematical reasoning (GSM8K [Cobbe et al. 2021]) and code generation (HumanEval [Chen et al. 2021]).

Pillar 4 — Multi-Metric Reporting. Accuracy or F1 alone is insufficient to characterize an APO method. APOBENCH requires six metrics reported jointly per optimizer–task pair: task performance (accuracy, F1, or Pass@1); total tokens consumed, as a proxy for API cost; wall-clock time, for latency and deployment feasibility; generations completed, for search depth within the budget; the stop reason, distinguishing convergence from budget exhaustion; and the standard deviation across seeds, for reliability.

Pillar 5 — Model Parity. All compared methods must use the same base LLM, identical decoding parameters, and the same zero-shot initial prompt (a minimal task description without worked examples). Differences in the base model render cross-method comparisons uninformative, as observed accuracy differences may reflect the model rather than the optimizer. In this study, all methods use Qwen3-4B-Instruct-2507 [Yang et al. 2025] under the shared decoding and search configuration detailed in Section 5.4.

The protocol is extensible by design. Any optimizer, any benchmark with a deterministic scorer and any locally hosted or API-served model can be added without modifying it.

5. Experimental Setup

5.1. Compared Optimizers

We evaluate seven gradient-free APO methods spanning the cost-behavior spectrum of the field: from token-efficient constrained search (DSGE-Evo, PLUM, HPSS) to high-cost evolutionary and critique-based methods (GAAPO, PromptBreeder, CriSPO), plus the hybrid SEE. Each is summarized by its core generation mechanism.

SEE [Cui et al. 2025] (Strategic Exploration and Exploitation) performs cohe-

sive in-context optimization through a quad-phased design alternating global exploration and local exploitation, jointly refining the instruction and its in-context examples; it is the only method here that converges before exhausting the budget. **GAAPO** [Sécheresse et al. 2025] (Genetic Algorithmic Applied to Prompt Optimization) maintains a population of prompts under LLM-guided crossover and mutation, selecting survivors by validation accuracy. **PromptBreeder** [Fernando et al. 2024] co-evolves task-prompts and the mutation-prompts that rewrite them, with tournament selection driving both populations.

CriSPO [He et al. 2025] critiques the current prompt across multiple quality dimensions and rewrites it; each iteration issues several sequential LLM calls, so it tends to exhaust the budget. **DSGE-Evo** [Saletta and Ferretti 2024] is our implementation of Saletta and Ferretti's grammar-based approach: candidates are sampled from a structured grammar and refined by mutations restricted to specific non-terminals, a constraint that avoids open-ended generation and makes it the most token-efficient method here. **PLUM** [Pan et al. 2024] applies discrete-optimization metaheuristics such as hill climbing, simulated annealing and genetic variants, converging quickly to a local optimum, and **HPSS** [Wen et al. 2025] combines a small evolutionary population with a local search heuristic. CriSPO and HPSS were both designed for open-ended generation and LLM-as-evaluator prompts respectively, rather than the reasoning tasks used here.

5.2. Benchmarks

Table 1 summarizes the three benchmarks, chosen to exercise distinct reasoning demands rather than to maximize coverage. BBH probes multi-step symbolic and commonsense reasoning, where the phrasing of an instruction can determine whether the model decomposes a problem correctly; GSM8K isolates arithmetic and chain-of-thought reasoning over grade-school word problems; and HumanEval shifts the target to program synthesis, where success is verified objectively by executing unit tests instead of by string matching. Together they span symbolic, mathematical, and executable reasoning, the three families most commonly reported in the APO literature. The BBH subset is aligned with the tasks evaluated in SEE [Cui et al. 2025] to facilitate comparison with the source literature, and the corresponding per-task results appear in Table 4.

Table 1. Benchmarks used in this study.

Benchmark	Size / Subset	Seeds	Metric
BBH [Suzgun et al. 2023]	8 out of 23 tasks	5	Exact-match
GSM8K [Cobbe et al. 2021]	1,319 problems	3	Exact-match
HumanEval [Chen et al. 2021]	164 problems	3	Pass@1

5.3. Implementation and Reproducibility Package

A protocol is reusable only if its implementation is. We release APOBENCH at `https://github.com/silveriorj/apobench`, with all seven optimizers behind one interface sharing the evaluation, budget-accounting and answer-extraction paths, so adding a method means implementing that interface rather than rebuilding a harness. A loader carves the test partition under a fixed split seed independent of the run seed, keeping the

held-out set identical across seeds and methods, and every run reported here is declared by a configuration file. The aggregation scripts implement Pillar 2's rules directly, computing means, standard deviations, paired tests and the Holm–Bonferroni correction, so a new optimizer can be scored against the published entries with cost beside accuracy.

5.4. Model and Budget

All experiments use **Qwen3-4B-Instruct-2507** [Yang et al. 2025] served locally via vLLM [Kwon et al. 2023] (NVIDIA RTX A4500, 20 GB VRAM, bf16 precision). The optimization budget combines three simultaneous caps per optimizer–task pair: 3,600 seconds of wall-clock time, 5,000 API calls, and 5,000,000 tokens. Whichever cap is reached first terminates the search. BBH is evaluated with five fixed random seeds (42, 13, 33, 27, 55); GSM8K and HumanEval use three (42, 123, 456). All methods receive the same minimal zero-shot initial prompt consisting solely of a task description. Each benchmark task uses a 30/70 validation/test split, with reported accuracy reflecting test-set performance only; means and standard deviations are computed per task.

To extend model parity (Pillar 5) to the search itself, all methods share a common configuration wherever their designs permit: generation cap 20, population size 10, mutation rate 0.9, crossover rate 0.5, elite size 3, with temperature 0.7 and top-p 0.9 (max. 256 new tokens) for candidate-generating calls and greedy decoding for evaluator calls. Method-specific parameters lacking counterparts elsewhere retain their published defaults, so observed differences reflect the search strategy rather than incidental tuning.

6. Results and Analysis

6.1. Accuracy Across Benchmarks

Table 2. Accuracy per benchmark (mean ± standard deviation across seeds). BBH uses five seeds; GSM8K and HumanEval use three. Bold = highest among optimizers; † = budget-exhausted lower bound.

Optimizer	BBH	GSM8K	HumanEval
Baseline	0.344±0.000	0.460±0.000	0.667±0.000
GAAPO†	**0.600**±0.007	**0.853**±0.061	**0.778**±0.018
SEE	0.586±0.009	0.837±0.006	**0.778**±0.018
CriSPO†	0.579±0.010	0.807±0.025	0.727±0.030
DSGE-Evo	0.579±0.015	0.807±0.059	0.758±0.000
PromptBreeder †	0.570±0.034	**0.853**±0.023	0.727±0.000
PLUM	0.558±0.011	0.600±0.020	0.758±0.000
HPSS	0.538±0.016	0.563±0.046	0.758±0.000

Every optimizer improves substantially over the unoptimized seed prompt, largest on BBH (baseline 0.344 vs. GAAPO 0.600) and narrower but consistent elsewhere, confirming that the gains reflect optimization rather than the base model's zero-shot ability. Against that reference, *no single method dominates all three benchmarks*: GAAPO leads on BBH and ties for first on GSM8K and HumanEval, yet PromptBreeder matches its GSM8K score while trailing on HumanEval, and PLUM and HPSS rank last on GSM8K while staying competitive elsewhere.

The variance figures tell a parallel story. SEE attains the lowest GSM8K standard deviation (± 0.006), while PromptBreeder shows zero variance on HumanEval, consistent with convergence to a fixed local optimum. The five-seed BBH run gives a clearer stability ranking: GAAPO and SEE are most stable (± 0.007, ± 0.009) and PromptBreeder least (± 0.034), its instability concentrated on the hardest tasks. Because rankings shift across benchmarks and seed variance can exceed the gaps between methods, any single-benchmark or single-seed evaluation risks mistaking a run-specific result for a general one.

6.2. Computational Efficiency

Table 3 presents efficiency metrics for the GSM8K task, which best discriminates methods on cost (HumanEval runs are shorter and less variable). Methods are sorted by ascending token consumption.

Table 3. Efficiency metrics on GSM8K (3-seed averages). Tokens reported in millions; Gens is the mean across seeds. Stop reasons: *early_stop* = no improvement for 5 consecutive generations (APOBENCH's patience criterion); *converged* = optimizer's own internal termination triggered; *budget* = any resource cap reached (wall-clock time or API call limit).

Optimizer	Tokens (M)	Time (s)	Gens	Stop Reason
PLUM	0.165	336	6.0	early_stop
DSGE-Evo	0.270	568	7.0	early_stop
HPSS	0.306	605	6.3	early_stop
SEE	0.822	1,844	2.0	converged
CriSPO	1.433	3,939	6.0	budget
PromptBreeder	1.552	3,542	8.0	budget
GAAPO	1.831	3,655	8.0	budget

The cost gap is wide: PLUM consumes $11\times$ fewer tokens than GAAPO, and DSGE-Evo searches $6.9\times$ faster than CriSPO (568 s versus 3,939 s). Yet this efficiency costs little accuracy. On GSM8K, DSGE-Evo matches CriSPO (0.807) at $5.3\times$ lower token cost; on BBH it approaches GAAPO (0.579 vs. 0.600) using roughly an order of magnitude fewer tokens per task, placing it on the Pareto front. On HumanEval, HPSS and PLUM come within one accuracy band of GAAPO (0.758 versus 0.778) at far lower cost.

6.3. Convergence Behavior and Stop Reason

The stop reason in Table 3 separates three convergence regimes: **converged** (SEE alone, its quad-phased design plateauing in about two generations); **early_stop** (PLUM, DSGE-Evo, HPSS, where the 5-generation patience criterion triggers well before the time cap); and **budget** (CriSPO, PromptBreeder, GAAPO, whose searches remain incomplete when the cap is reached, making their scores lower bounds). This last regime never appears in an accuracy table on its own, which is why APOBENCH makes the stop reason mandatory.

The returned prompts corroborate this reading. Their lengths on GSM8K span a factor of 32 and sort almost exactly by convergence regime. The four methods that stop

early or converge return the shortest prompts, at 54 characters for PLUM, 89 for DSGE-Evo, 124 for SEE and 216 for HPSS, while the three that exhaust their budget return the longest, at 740 for PromptBreeder, 1,069 for GAAPO and 1,756 for CriSPO. Failing to converge therefore costs more than time. It also leaves the optimizer accreting text.

Prompt content divides along the same line. PLUM returns a bare directive, *"The following math problem can be solved step by step."* SEE, the only method that converges, constrains the output format as well, instructing the model to give only the final numerical answer and to avoid any explanation. CriSPO's prompt has instead absorbed an entire evaluation rubric, with numbered quality dimensions and worked arithmetic traces carried over from training problems. Length, however, does not buy accuracy. SEE's 124-character prompt scores 0.837 on GSM8K, above the 0.807 obtained by CriSPO's far longer one. Since the returned prompt is prepended to every query the deployed system later serves, its length is a recurring inference cost that accuracy-only comparisons leave out of view.

6.4. Task-Level Stability on BBH

Table 4 presents per-task accuracy on the 8 BBH tasks from the 5-seed run. Each cell shows the mean across 5 seeds; the rightmost column is the per-optimizer mean across all 8 tasks.

Table 4. Per-task accuracy on BBH. Optimizer cells show the mean (standard deviation) across five seeds; the baseline is a fixed, unoptimized prompt. Bold marks the highest mean per task among optimizers. CJ = causal_judgement, DQ = disambiguation_qa, DL = dyck_languages, FF = formal_fallacies, HY = hyperbaton, LD = logical_deduction_5obj, RC = reasoning_colored_objects, ST = salient_translation.

Opt.	CJ	DQ	DL	FF	HY	LD	RC	ST	Avg
Baseline	0.121	0.630	0.030	0.290	0.677	0.070	0.363	0.573	0.344
CriSPO	**0.645** (0.026)	0.510 (0.054)	0.414 (0.048)	0.632 (0.042)	0.860 (0.034)	0.476 (0.055)	0.558 (0.018)	0.538 (0.010)	0.579 (0.010)
DSGE-Evo	0.570 (0.029)	**0.580** (0.080)	0.386 (0.037)	0.640 (0.049)	0.770 (0.031)	**0.540** (0.032)	0.574 (0.051)	0.570 (0.041)	0.579 (0.015)
GAAPO	0.589 (0.062)	0.550 (0.054)	**0.448** (0.047)	0.632 (0.044)	**0.880** (0.022)	0.524 (0.023)	0.614 (0.028)	0.560 (0.032)	**0.600** (0.007)
HPSS	0.566 (0.046)	0.458 (0.039)	0.338 (0.016)	0.586 (0.054)	0.688 (0.046)	0.490 (0.017)	0.610 (0.037)	0.570 (0.041)	0.538 (0.016)
PLUM	0.557 (0.037)	0.508 (0.080)	0.326 (0.020)	0.618 (0.042)	0.752 (0.036)	0.530 (0.025)	0.608 (0.054)	0.566 (0.038)	0.558 (0.011)
PB	0.568 (0.131)	0.574 (0.077)	0.264 (0.137)	0.562 (0.040)	0.856 (0.041)	0.530 (0.040)	**0.634** (0.047)	0.572 (0.042)	0.570 (0.034)
SEE	0.604 (0.047)	0.466 (0.039)	0.416 (0.047)	**0.658** (0.038)	0.870 (0.029)	0.494 (0.015)	0.606 (0.044)	**0.574** (0.045)	0.586 (0.009)

Among BBH tasks, *hyperbaton* yields the highest scores across all methods (0.688–0.880), while *dyck_languages* is the most difficult, with GAAPO peaking at 0.448 and PromptBreeder dropping to 0.264. The per-task baseline clarifies where optimization contributes most. On *dyck_languages* and *logical_deduction_5obj* the seed prompt scores only 0.030 and 0.070, so every optimizer multiplies accuracy several times over, whereas on *disambiguation_qa* the baseline already reaches 0.630 and the optimizers add little.

Per-task rankings are inconsistent: CriSPO ranks first on *causal_judgement* yet trails DSGE-Evo on *disambiguation_qa*; DSGE-Evo leads on two tasks but ranks lower on *hyperbaton*. Task type substantially affects relative performance across optimizers.

Pillar 2's comparison rules turn that observation into a formal result. Every optimizer beats the baseline by a wide margin, with mean differences from $+0.194$ to $+0.255$ and p between 0.011 and 0.043, so optimization clearly helps. Almost nothing, however,

separates the optimizers from each other. Of the 21 pairwise comparisons, only GAAPO against HPSS reaches $p < 0.05$ uncorrected ($t(7) = 2.60$, $p = 0.035$), and once Holm–Bonferroni is applied no pairwise difference survives at all. The ranking implied by the Avg column is therefore not statistically supported at this seed budget, and cost becomes the axis on which these methods can actually be told apart.

7. Discussion

The multi-benchmark, multi-metric view exposes an efficiency–accuracy frontier rather than a single best method. GAAPO, SEE and PromptBreeder achieve task-dependent gains at substantially higher token and time cost, while PLUM, DSGE-Evo and HPSS sit close to that frontier at a fraction of the cost. Which trade-off is preferable depends on the task. On BBH the gap between DSGE-Evo and GAAPO is negligible and the cheaper method dominates, whereas on GSM8K the spread from PLUM (0.600) to GAAPO and PromptBreeder (0.853) justifies the added search cost. Section 6.4 sharpens this reading. Once variance and multiple comparisons are accounted for, the accuracy ordering among optimizers is not statistically supported at all, which leaves cost and convergence behavior as the axes that genuinely separate these methods.

GAAPO and PromptBreeder also exhaust the budget after only one or two generations in some runs, so their GSM8K accuracy is a lower bound that no accuracy metric can detect. For that reason APOBENCH treats the stop reason as a required field.

The practical reading follows. When a token or latency budget binds, a constrained method such as DSGE-Evo offers most of the accuracy of the heavier optimizers at a fraction of the cost; when resources are ample and accuracy on a hard reasoning benchmark is paramount, an evolutionary method such as GAAPO is the stronger choice, provided its budget-exhausted scores are read as lower bounds. Only a protocol that fixes the budget, controls seeds, spans multiple benchmarks, and reports cost alongside accuracy can surface this trade-off.

Limitations. Our study rests on a single 4B-parameter model and English-only benchmarks, with an eight-task subset representing BBH, so rankings may shift under larger models, other languages or different budgets. The protocol carries further caveats. Its wall-clock cap is hardware-dependent and less portable than a token budget, its deterministic-scorer requirement rules out open-ended generation, and its baseline is a single zero-shot prompt rather than a human-tuned reference, which lower-bounds achievable performance without establishing an upper bound.

8. Conclusion

We introduced APOBENCH, a five-pillar evaluation protocol for fair and reproducible comparison of Automatic Prompt Optimization methods. Its five pillars of budget standardization, seed control, benchmark diversity, multi-metric reporting and model parity formalize decisions that the APO literature leaves implicit, enabling head-to-head comparison that existing work does not support.

Validating it across seven gradient-free methods on 10 tasks, we found that every method beat an unoptimized baseline, yet none led on every benchmark and the accuracy leaders were not the cost leaders. Under paired testing with correction, no pairwise accuracy difference between optimizers is significant at all, which leaves cost and convergence

as the axes that separate them. The stop reason, overlooked in prior comparisons, showed that the three highest-cost methods never completed their searches, making their accuracy a lower bound.

References

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Cui, W., Zhang, J., Li, Z., Sun, H., Lopez, D., Das, K., Malin, B. A., and Kumar, S. (2025). SEE: Strategic exploration and exploitation for cohesive in-context prompt optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), Volume 1: Long Papers*, pages 29575–29627, Vienna, Austria.

Fernando, C., Banarse, D., Michalewski, H., Osindero, S., and Rocktäschel, T. (2024). Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235, pages 13481–13544.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. (2024). A framework for few-shot language model evaluation.

Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., and Yang, Y. (2024). Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

He, H., Liu, Q., Xu, L., Shivade, C., Zhang, Y., Srinivasan, S., and Kirchhoff, K. (2025). CriSPO: Multi-aspect critique-suggestion-guided automatic prompt optimization for

text generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24014–24022.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. (2024). DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*.

Li, W., Wang, X., Li, W., and Jin, B. (2025). A survey of automatic prompt engineering: An optimization perspective. *arXiv preprint arXiv:2502.11560*.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D., Hudson, D. A., Zelikman, E., Durmus, E., Ladhak, F., Rong, F., Ren, H., Yao, H., Wang, J., Santhanam, K., Orr, L., Zheng, L., Yuksekgonul, M., Suzgun, M., Kim, N., Guha, N., Chatterji, N., Khattab, O., Henderson, P., Huang, Q., Chi, R., Xie, S. M., Santurkar, S., Ganguli, S., Hashimoto, T., Icard, T., Zhang, T., Chaudhary, V., Wang, W., Li, X., Mai, Y., Zhang, Y., and Koreeda, Y. (2023). Holistic evaluation of language models. *Transactions on Machine Learning Research*.

Mizrahi, M., Kaplan, G., Malkin, D., Dror, R., Shahaf, D., and Stanovsky, G. (2024). State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949.

Pan, R., Xing, S., Diao, S., Sun, W., Liu, X., Shum, K., Zhang, J., Pi, R., and Zhang, T. (2024). PLUM: Prompt learning using metaheuristics. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2177–2197.

Pryzant, R., Iter, D., Li, J., Lee, Y. T., Zhu, C., and Zeng, M. (2023). Automatic prompt optimization with "Gradient Descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pages 7957–7968, Singapore.

Ramnath, K. et al. (2025). A systematic survey of automatic prompt optimization techniques. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025), Main*, pages 33078–33110, Suzhou, China.

Saletta, M. and Ferretti, C. (2024). Exploring the prompt space of large language models through evolutionary sampling. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '24)*, pages 1345–1353.

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Sécheresse, X., Guilbert-Ly, J.-Y., and Villedieu de Torcy, A. (2025). GAAPO: Genetic algorithmic applied to prompt optimization. *Frontiers in Artificial Intelligence*, 8. Also presented at ECAI 2025.

Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. (2023). Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, Toronto, Canada.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., and Fedus, W. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*.

Wen, B., Ke, P., Sun, Y., Wang, C., Gu, X., Zhou, J., Tang, J., Wang, H., and Huang, M. (2025). HPSS: Heuristic prompting strategy search for LLM evaluators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24974–25007, Vienna, Austria.

Yang, A. et al. (2025). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. (2024). Large language models as optimizers. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Zehle, T., Heiß, T., Schlager, M., Aßenmacher, M., and Feurer, M. (2026). promptolution: A unified, modular framework for prompt optimization. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 282–296, Rabat, Morocco.

Zehle, T., Schlager, M., Heiß, T., and Feurer, M. (2025). CAPO: Cost-aware prompt optimization. In *Proceedings of the Fourth International Conference on Automated Machine Learning*, volume 293 of *Proceedings of Machine Learning Research*.

Zheng, W., Das, S. S. S., Zhang, Y., and Zhang, R. (2025). GreaterPrompt: A unified, customizable, and high-performing open-source toolkit for prompt optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025): System Demonstrations*.

Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., and Ba, J. (2023). Large language models are human-level prompt engineers. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*.

Zhu, K., Wang, J., Zhou, J., Wang, Z., Chen, H., Wang, Y., Yang, L., Ye, W., Gong, N. Z., Zhang, Y., and Xie, X. (2023). PromptBench: Towards evaluating the robustness of large language models on adversarial prompts. *arXiv preprint arXiv:2306.04528*.

Zhuo, J., Zhang, S., Fang, X., Duan, H., Lin, D., and Chen, K. (2024). ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1950–1976, Miami, Florida, USA.